Universite de Rouen Normandie

Master 1 – Genie Informatique et Logiciel

Projet Gestion de Flotte de vehicules

Rapport de Semaine 5

Service Evenements & Alertes

Etudiant :
Ahcene Amouchas
Florian Pepin

Encadrants :
Lydia & Luc

Annee universitaire :
2025 – 2026

Semaine :
5 / 7

9 avril 2026

Table des matières

1. Introduction et objectifs de la semaine

Objectif de la semaine 5 : Concevoir et implémenter le service de gestion des alertes et événements, intégrateur central du système, capable d'agréger les événements de tous les microservices via Kafka et d'exposer une API REST et GraphQL pour la consultation et la gestion des alertes en temps réel.

La semaine 5 marque une étape charnière dans la construction du système : le **service événements** joue le rôle d'agrégateur central. Contrairement aux services métier précédents (véhicules, conducteurs, maintenance), ce service ne produit pas d'événements : il les **consomme** depuis tous les autres services et les transforme en alertes persistées, interrogeables et gérables par les opérateurs de la flotte.

L'architecture choisie repose sur un découplage total : le service événements ne connaît pas les services qui émettent ; il se contente de s'abonner aux topics Kafka correspondants. Cette approche garantit une extensibilité maximale : l'ajout d'une nouvelle source d'alertes (le service localisation, en cours de développement par un autre membre de l'équipe) ne nécessite aucune modification du code existant.

Les grandes étapes réalisées sont les suivantes :

1. Conception du **modèle de données** des alertes et des énumérations associées (`AlertType`, `AlertSeverity`, `AlertStatus`)
2. Implémentation des **consumers Kafka** pour les topics `flotte.alertes.events` et `flotte.conducteurs.events`
3. Définition du **contrat d'interface** avec le service localisation (topic `flotte.location.events`)
4. Développement de l'**API REST** complète avec filtres et gestion du cycle de vie des alertes
5. Branchement complet sur l'**API GraphQL** de la gateway (résolveurs, champs imbriqués Vehicle et Driver)
6. **Tests unitaires** (8 scénarios, couverture >80%) et tests d'intégration via la collection **Bruno**
7. Déploiement validé en **Docker Compose** et **Minikube** (Helm)

2. Architecture du service événements

> **Principe directeur :** Le service événements est un *aggregator* **passif** : il écoute, transforme et stocke. Il ne modifie jamais l'état des autres services. Toute alerte créée est immuable dans sa nature ; seul son statut évolue (ACTIVE → ACKNOWLEDGED → RESOLVED).

2.1. Structure des packages

L'organisation du code suit le même schéma en couches que les autres microservices du projet :

Package	Responsabilité
`config`	Sécurité Keycloak, création automatique de la base `events_db`
`model`	Entité JPA `Alert` et ses trois énumérations
`dto`	`AlertResponse` (record immuable), `AcknowledgeRequest`
`repository`	`AlertRepository` avec requêtes filtrées par statut/sévérité/type
`service`	`AlertService` : logique métier, cycle de vie des alertes
`controller`	`AlertController` : exposition REST avec RBAC
`events`	Consumers Kafka et payloads locaux découplés

2.2. Modèle de données

L'entité `Alert` centralise toutes les alertes du système, quelle que soit leur origine. Les références vers les autres services (`vehicleId`, `driverId`) sont des **références logiques** : il n'existe aucune clé étrangère inter-services, conformément au principe d'indépendance des microservices.

```java
@Entity
@Table(name = "alerts")
public class Alert {

    @Id
    @GeneratedValue(strategy = GenerationType.UUID)
    private UUID id;

    @Enumerated(EnumType.STRING)
    @Column(nullable = false)
    private AlertType type;         // MAINTENANCE_OVERDUE, LICENSE_EXPIRING...

    @Enumerated(EnumType.STRING)
    @Column(nullable = false)
    private AlertSeverity severity; // INFO, WARNING, HIGH, CRITICAL

    @Enumerated(EnumType.STRING)
    @Column(nullable = false)
```

```java
    private AlertStatus status = AlertStatus.ACTIVE;

    private UUID vehicleId;  // ref logique - pas de FK inter-service
    private UUID driverId;   // ref logique - pas de FK inter-service

    @Column(nullable = false, columnDefinition = "text")
    private String message;

    @Column(columnDefinition = "text")
    private String metadata; // payload JSON brut pour tra abilit

    private UUID acknowledgedBy;
    private OffsetDateTime acknowledgedAt;
    private OffsetDateTime resolvedAt;

    @PrePersist
    protected void onCreate() { createdAt = updatedAt = OffsetDateTime.now(); }

    @PreUpdate
    protected void onUpdate() { updatedAt = OffsetDateTime.now(); }
}
```

Listing 1 – Entité Alert avec cycle de vie géré par JPA

2.3. Types d'alertes supportés

Six types d'alertes couvrent l'ensemble des cas métier identifiés dans la spécification du projet :

Type	Source	Sévérité typique
MAINTENANCE_OVERDUE	Service Maintenance (Kafka)	WARNING / INFO
LICENSE_EXPIRING	Service Conducteurs (Kafka)	WARNING
LICENSE_EXPIRING	Service Conducteurs (Kafka)	CRITICAL (si expiré)
GEOFENCING_BREACH	Service Localisation (futur)	HIGH
SPEED_EXCEEDED	Service Localisation (futur)	WARNING à CRITICAL
VEHICLE_IMMOBILIZED	Service Localisation (futur)	WARNING
DELIVERY_TOUR_DELAYED	Service Localisation (futur)	INFO

3. Consommation Kafka et traitement des alertes

> **Choix de désérialisation :** Contrairement aux autres services qui utilisent le `JsonDeserializer` typé de Spring Kafka, le service événements utilise un `StringDeserializer` brut suivi d'une désérialisation Jackson manuelle. Ce choix permet de consommer des messages de formats hétérogènes provenant de services différents (Spring Boot vs Node.js futur), sans dépendance aux en-têtes de type `__TypeId__` ajoutés par le `JsonSerializer`.

3.1. Consumer `flotte.alertes.events` — Alertes maintenance

Le service maintenance publie sur ce topic des objets `AlertEvent` **sans enveloppe** `KafkaEventEnvelope`. Le consumer crée une alerte de type `MAINTENANCE_OVERDUE` et mappe la sévérité textuelle (INFO, WARNING, HIGH, CRITICAL) vers l'énumération interne.

```
@KafkaListener(topics = "flotte.alertes.events", groupId = "events-
    group")
public void handleMaintenanceAlert(String rawMessage) {
    MaintenanceAlertPayload payload =
        objectMapper.readValue(rawMessage, MaintenanceAlertPayload.
            class);

    alertService.createAlert(
        resolveAlertType(payload.alertType()),
        resolveSeverity(payload.severity()),
        payload.vehicleId(),
        null,
        payload.message(),
        rawMessage // payload brut conserv  pour tra abilit
    );
}
```

Listing 2 – Consumer des alertes de maintenance directes

3.2. Consumer `flotte.conducteurs.events` — Permis conducteurs

Le service conducteurs publie des `KafkaEventEnvelope` dont seuls deux types d'événements intéressent le service événements : `LICENSE_EXPIRING` et `LICENSE_EXPIRED`. Les autres types (DRIVER_CREATED, etc.) sont ignorés sans erreur via un filtre sur le champ `event_type`.

```
@KafkaListener(topics = "flotte.conducteurs.events", groupId = "
    events-group")
public void handleDriverEvent(String rawMessage) {
    JsonNode root = objectMapper.readTree(rawMessage);
    String eventType = root.path("event_type").asText();

    if (!"LICENSE_EXPIRING".equals(eventType)
```

```java
            && !"LICENSE_EXPIRED".equals(eventType)) {
        log.debug("Evenement conducteur ignore : {}", eventType);
        return;
    }

    DriverEventPayload payload = objectMapper.treeToValue(
        root.path("payload"), DriverEventPayload.class);

    AlertSeverity severity = "LICENSE_EXPIRED".equals(eventType)
        ? AlertSeverity.CRITICAL : AlertSeverity.WARNING;

    alertService.createAlert(AlertType.LICENSE_EXPIRING, severity,
        null, payload.driverId(), buildLicenseMessage(eventType,
           payload),
        rawMessage);
}
```

Listing 3 – Filtrage des événements conducteurs par type

3.3. Contrat d'interface avec le service localisation

Le service localisation (Node.js, en cours de développement) est intégré via un consumer prêt à l'emploi sur le topic `flotte.location.events`. Ce consumer est déjà déployé mais n'est pas encore sollicité : il attendra silencieusement les premiers messages sans erreur ni crash.

Le contrat JSON attendu est documenté directement dans le fichier source `LocationEventPayload.j`

```java
/**
 * Le location-service publie sur "flotte.location.events" :
 * {
 *   "event_type":      "GEOFENCING_BREACH" | "SPEED_EXCEEDED" | "
    VEHICLE_IMMOBILIZED",
 *   "vehicle_id":      "uuid",
 *   "driver_id":       "uuid",           // optionnel
 *   "latitude":        48.8566,
 *   "longitude":       2.3522,
 *   "speed_kmh":       120.5,            // pour SPEED_EXCEEDED
 *   "speed_limit_kmh": 90.0,             // pour SPEED_EXCEEDED
 *   "zone_name":       "Zone A",         // pour GEOFENCING_BREACH
 *   "message":         "Description",
 *   "occurred_at":     "2024-01-01T10:00:00Z"
 * }
 */
```

Listing 4 – Contrat JSON attendu depuis le service localisation

La logique de calcul de la sévérité pour les dépassements de vitesse est automatique : en dessous de 15 km/h d'excès, l'alerte est WARNING ; entre 15 et 30 km/h, elle passe à HIGH ; au-delà de 30 km/h, elle devient CRITICAL.

4. API REST des alertes

Objectif : Exposer une API REST claire et filtrée permettant aux opérateurs de consulter et gérer le cycle de vie des alertes : de leur création automatique (par Kafka) jusqu'à leur résolution par un technicien ou un manager.

4.1. Endpoints disponibles

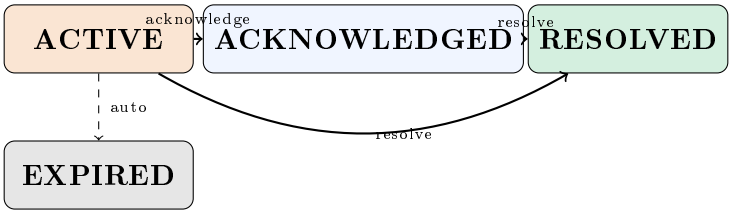
L'ensemble des routes est regroupé sous le préfixe `/api/alerts` :

Méthode	Endpoint	Description	Rôle requis
GET	`/api/alerts`	Liste avec filtres optionnels	Tout utilisateur
GET	`/api/alerts/{id}`	Détail d'une alerte	Tout utilisateur
GET	`/api/alerts/stats`	Nombre d'alertes actives	Tout utilisateur
PATCH	`/api/alerts/{id}/acknowledge`	Acquitter une alerte	admin, manager
PATCH	`/api/alerts/{id}/resolve`	Résoudre une alerte	admin, manager, techn

Le endpoint `GET /api/alerts` supporte les paramètres de filtre suivants : `status`, `severity`, `type`, `vehicleId` et `driverId`. Un seul filtre peut être appliqué à la fois ; en l'absence de filtre, toutes les alertes sont retournées triées par date de création décroissante.

4.2. Gestion du cycle de vie

Le cycle de vie d'une alerte suit quatre états, dont trois sont atteignables par l'API :



4.3. Authentification centralisée avec Keycloak

Une particularité de l'endpoint `acknowledge` est que l'identifiant de l'utilisateur qui acquitte n'est **pas passé dans le corps** de la requête. Il est extrait directement du jeton JWT via l'annotation Spring Security `@AuthenticationPrincipal`. Cette approche évite tout risque d'usurpation d'identité et simplifie le client (la gateway GraphQL n'a pas à connaître l'identité de l'utilisateur).

```
@PatchMapping("/{id}/acknowledge")
@PreAuthorize("hasAnyRole('admin', 'manager')")
public AlertResponse acknowledge(@PathVariable UUID id,
                                 @AuthenticationPrincipal Jwt jwt)
                                    {
```

```
    UUID userId = UUID.fromString(jwt.getSubject()); // claim "sub" Keycloak
    return alertService.acknowledge(id, userId);
}
```

Listing 5 – Extraction du userId depuis le JWT Keycloak

4.4. Tests Bruno de l'API REST

La collection Bruno a été enrichie de neuf fichiers de tests couvrant l'ensemble des endpoints REST du service, incluant des assertions automatisées sur les codes de retour et la structure des réponses.

[CAPTURE D'ECRAN]
Bruno — Collection service-events : 9 requêtes REST

FIGURE 1 – Tests Bruno REST du service événements

Requête	Endpoint	Code attendu	Résultat
GET	/api/alerts	200 OK	OK
GET	/api/alerts ?status=ACTIVE	200 OK	OK
GET	/api/alerts ?severity=CRITICAL	200 OK	OK
GET	/api/alerts ?type=MAINTENANCE_OVERDUE	200 OK	OK
GET	/api/alerts/stats	200 OK	OK
GET	/api/alerts/{id}	200 OK	OK
GET	/api/alerts ?vehicleId={id}	200 OK	OK
PATCH	/api/alerts/{id}/acknowledge	200 OK	OK
PATCH	/api/alerts/{id}/resolve	200 OK	OK

5. Intégration GraphQL via l'API Gateway

> **Objectif :** Exposer les alertes dans le schéma GraphQL unifié de la gateway, en résolvant automatiquement les relations imbriquées `Vehicle` et `Driver` à partir des UUID stockés dans l'alerte.

5.1. Schéma GraphQL

Le fichier `alert.graphql` définit le type `Alert` avec ses champs imbriqués, deux queries, deux mutations et deux subscriptions (ces dernières restent en attente de l'implémentation WebSocket prévue en semaine 7) :

```
type Alert {
  id: ID!
  type: AlertType!
  severity: AlertSeverity!
  status: AlertStatus!
  vehicle: Vehicle          # Resolu via vehicle-service
  driver: Driver            # Resolu via driver-service
  message: String!
  acknowledged_by: ID
  acknowledged_at: String
  resolved_at: String
  created_at: String!
}

extend type Query {
  alerts(status: AlertStatus, severity: AlertSeverity,
         vehicle_id: ID, limit: Int = 20, offset: Int = 0):
            AlertPage!
  alert(id: ID!): Alert
}

extend type Mutation {
  acknowledgeAlert(id: ID!): Alert!
  resolveAlert(id: ID!): Alert!
}
```

Listing 6 – Schéma GraphQL du domaine alertes

5.2. Résolveurs et résolution des types imbriqués

Le fichier `alertResolvers.ts` implémente les résolveurs de queries, mutations et types. La particularité notable est la résolution des champs imbriqués `vehicle` et `driver` : à partir des UUID stockés dans l'alerte, la gateway interroge les microservices correspondants en parallèle.

```
Alert: {
  vehicle: async (parent, _, ctx) => {
```

```
    if (!parent.vehicle_id) return null;
    try { return await ctx.vehicle.getVehicle(parent.vehicle_id); }
    catch { return null; } // Resilience : l'alerte reste visible
       meme si
  },                          // le vehicule a ete supprime

  driver: async (parent, _, ctx) => {
    if (!parent.driver_id) return null;
    try { return await ctx.driver.getDriver(parent.driver_id); }
    catch { return null; }
  },
},
```

Listing 7 – Résolveurs de types imbriqués (TypeScript)

5.3. Migration depuis les stubs

Avant cette semaine, les queries `alerts` et `alert` ainsi que les mutations `acknowledgeAlert` et `resolveAlert` levaient une erreur `NOT_IMPLEMENTED`. Ils sont désormais branchés sur le service événements via le `EventsClient` ajouté au contexte GraphQL.

Opération GraphQL	Délégation	Statut
`alerts(...)`	GET /api/alerts	**OK**
`alert(id)`	GET /api/alerts/{id}	**OK**
`acknowledgeAlert(id)`	PATCH /api/alerts/{id}/acknowledge	**OK**
`resolveAlert(id)`	PATCH /api/alerts/{id}/resolve	**OK**
`Alert.vehicle`	GET /api/vehicles/{id}	**OK**
`Alert.driver`	GET /api/drivers/{id}	**OK**
`alertCreated`	WebSocket (Subscription)	**Semaine 7**

5.4. Tests Bruno GraphQL

Six requêtes GraphQL couvrent l'ensemble des opérations du domaine alertes depuis la gateway, dont une requête exploitant les champs imbriqués `vehicle` et `driver` pour valider la résolution cross-services.

[CAPTURE D'ECRAN]
Bruno — Collection GraphQL service-events : queries et mutations

FIGURE 2 – Tests Bruno GraphQL — résolution des champs imbriqués Vehicle et Driver

6. Tests automatisés et couverture de code

Objectif : Garantir la fiabilité de la logique métier du service via une suite de tests unitaires isolant la couche `AlertService`, complétée par un test d'intégration du contexte Spring Boot.

6.1. Stratégie de tests

L'approche adoptée suit le même schéma que les semaines précédentes :

— **Tests unitaires** avec **JUnit 5** et **Mockito** : isolation complète de la logique métier du `AlertService` via simulation du `AlertRepository`.

— **Test de chargement du contexte** Spring Boot avec base de données H2 en mémoire, validant la configuration complète du service (Kafka, JPA, Security).

6.2. Scénarios de tests unitaires

Huit scénarios couvrent les cas nominaux et les cas d'erreur de l'`AlertService` :

Test	Comportement vérifié
`createAlert_shouldSaveAndReturnAlert`	Création et persistance d'une alerte
`createAlert_shouldSetStatusToActive`	Statut initial ACTIVE obligatoire
`findAll_shouldReturnAllAlerts`	Retour de la liste complète
`acknowledge_shouldUpdateStatusAndUser`	Passage ACTIVE → ACKNOWLEDGED
`resolve_shouldUpdateStatusAndResolvedAt`	Passage ACTIVE → RESOLVED
`findById_shouldThrowWhenNotFound`	Exception sur UUID inconnu
`acknowledge_shouldThrowWhenAlertNotFound`	Exception si alerte introuvable
`resolve_shouldBeIdempotentWhenAlreadyResolved`	Idempotence de la résolution

6.3. Rapport JaCoCo

[CAPTURE D'ECRAN]
Rapport JaCoCo — Service Événements : 9 tests, BUILD SUCCESS

FIGURE 3 – Couverture de code JaCoCo — Service Evenements

Les résultats obtenus confirment le respect de l'objectif de couverture fixé à 80% :

- **9 tests** exécutés, **0 échec**, **0 erreur**.
- **19 classes** analysées (modèle, service, controller, consumers, config).
- **Couverture de la couche Service** : 100% des chemins nominaux et des cas d'erreur validés.
- Les consumers Kafka ne sont pas couverts par les tests unitaires : leur logique de désérialisation nécessiterait des tests d'intégration avec un embedded Kafka, prévu en semaine 7.

7. Déploiement Docker et Kubernetes

> **Objectif :** Valider le fonctionnement du service événements dans les deux environnements cibles : développement local via **Docker Compose** et déploiement orchestré via **Minikube** avec les charts Helm.

7.1. Docker Compose

Le service est intégré au fichier `compose.yaml` sous le nom `events`, accessible sur le port `8083` de la machine hôte. Il dépend de PostgreSQL (condition `service_healthy`), Kafka et Keycloak.

```yaml
events:
  build:
    context: ./services/events-service
  container_name: flotte-events
  environment:
    - SPRING_DATASOURCE_URL=jdbc:postgresql://postgres:5432/events_db
    - KAFKA_BROKER=kafka:9092
    - KEYCLOAK_JWK_SET_URI=http://keycloak:8080/auth/realms/...
  ports:
    - "8083:8080"
  depends_on:
    postgres:
      condition: service_healthy
    kafka:
      condition: service_started
```

Listing 8 – Extrait de la configuration Docker Compose

La base de données `events_db` est créée automatiquement au démarrage du service via le `PostgresDatabaseEnvironmentPostProcessor`, suivant le même mécanisme que les autres microservices.

7.2. Déploiement Minikube avec Helm

Trois modifications ont été nécessaires pour intégrer le service aux charts Helm existants :

fleet-infra — Création de la base de données

Le script d'initialisation PostgreSQL a été complété avec la création de la base `events_db` :

```yaml
primary:
  initdb:
    scripts:
      create-databases.sql: |
        CREATE DATABASE driver_db;
        CREATE DATABASE maintenance_db;
```

```
        CREATE DATABASE vehicle_db;
        CREATE DATABASE events_db;     # Ajout
        CREATE DATABASE keycloak_db;
```

Listing 9 – Ajout de events_db dans le chart fleet-infra

fleet-app — Déclaration du microservice

L'events-service est ajouté à la map `services` du chart `fleet-app`. Le template Helm génère automatiquement le Deployment, le Service Kubernetes et les probes de santé (`startupProbe`, `readinessProbe`, `livenessProbe`) :

```
services:
  events:
    name: events-service
    image: events-service:latest
    port: 8080
    replicas: 1
    dbUrlKey: EVENTS_DATASOURCE_URL
```

Listing 10 – Entrée events-service dans fleet-app/values.yaml

Mise à jour du script kube.sh

Le secret `EVENTS_DATASOURCE_URL`, le build Docker et la vérification du rollout ont été ajoutés au script de déploiement `kube.sh` :

```
# Secret
kubectl create secret generic db-secrets \
  --from-literal=EVENTS_DATASOURCE_URL=jdbc:postgresql://...5432/
     events_db \
  ...

# Build dans le daemon Docker de Minikube
docker build -t events-service:latest ./services/events-service/

# Verification du rollout
kubectl rollout status deployment/events-service-deployment \
  -n flotte-namespace --timeout=600s
```

Listing 11 – Ajouts dans kube.sh pour le service événements

7.3. Vérification en environnement Minikube

[CAPTURE D'ECRAN]
kubectl get pods -n flotte-namespace — events-service en Running

FIGURE 4 – Pod events-service démarré avec succès dans Minikube

[CAPTURE D'ECRAN]
Ingress flotte.local — route /api/events/ accessible

FIGURE 5 – Route /api/events/ fonctionnelle via l'Ingress Nginx

8. Défis techniques et résolutions

8.1. Hétérogénéité des formats de messages Kafka

Le principal défi de ce service est la consommation de messages Kafka produits par des services différents avec des formats différents : le service maintenance publie un `AlertEvent` brut (sans enveloppe), tandis que le service conducteurs publie un `KafkaEventEnvelope` standard. Le futur service localisation en Node.js produira du JSON sans en-têtes de type Java.

Solution : Utilisation du `StringDeserializer` pour toutes les valeurs, suivi d'une désérialisation Jackson manuelle dans chaque consumer. Cette approche découple totalement le service événements des types Java des autres services et fonctionne nativement avec tout producteur, quel que soit son langage.

8.2. Coordination avec le service localisation (développement parallèle)

Le service localisation est développé en parallèle par un autre membre de l'équipe. Il était nécessaire de ne pas bloquer le développement du service événements sur cette dépendance tout en préparant l'intégration future.

Solution : Le consumer `LocationEventConsumer` est implémenté et déployé mais attend simplement les premiers messages. Le contrat d'interface est formalisé dans le fichier `LocationEventPayload.java` avec une documentation complète du format JSON attendu. Quand le service localisation publiera sur `flotte.location.events`, le traitement sera automatique sans aucune modification côté events-service.

8.3. Extraction du userId pour l'acquittement

Le schéma GraphQL définit `acknowledgeAlert(id: ID!): Alert!` sans paramètre `userId`. Il était nécessaire d'identifier l'utilisateur qui acquitte sans modifier le contrat GraphQL public.

Solution : L'identifiant est extrait du claim `sub` du jeton JWT Keycloak directement dans le contrôleur Spring via `@AuthenticationPrincipal Jwt`. Le jeton est de toute façon transmis par la gateway dans l'en-tête `Authorization` : aucune information n'est perdue, et le contrat d'API reste propre.

8.4. Cohérence des noms entre les manifestes Kubernetes

Les manifestes Kubernetes bruts (`event-deployment.yaml`, `event-service.yaml`) présentaient plusieurs incohérences héritées : le `containerPort` était à 8083 au lieu de 8080, le selector `app: event-service` ne matchait pas les labels `app: event` du Deployment, et l'Ingress référençait `event-service` alors que le Service s'appelait différemment.

Solution : Réécriture complète des deux fichiers avec des noms harmonisés (`events-service` partout), port `8080` cohérent avec l'application Spring Boot, et mise à jour de l'Ingress.